EF234405 — Design & Analysis of Algorithms
Final Exam — Group Capstone Project

SIGAP: Fastest-Route Emergency Dispatch on City Road Networks

Dijkstra vs. Bellman-Ford vs. A — Design, Proof, Build, Measure*

Team members
Muhammad Rizal Hafiyyan — 5025231212

Class: E

Surabaya, 18 June 2026

GitHub repository
https://github.com/0ryzal/EAS-PAA-E

Abstract — We model emergency-vehicle routing as a single-pair shortest-path problem on a weighted road graph and solve it with three algorithms implemented from scratch: Dijkstra (core), Bellman-Ford (baseline oracle) and A (informed). We prove Dijkstra optimal, derive the time and space complexity of each, and benchmark them across five network sizes from 100 to 10,000 intersections. All three return the identical optimal route on every instance; measured growth (Dijkstra ~ n^1.11, A* ~ n^1.02, Bellman-Ford ~ n^1.43) matches the theory.*

1 Design

1.1 Problem statement & motivation (D1)

When a 119 ambulance is dispatched, **every second counts**: survival from cardiac arrest falls by roughly 7–10% per minute without intervention. A dispatcher must, in real time, choose the **fastest** route from the nearest station to the incident over a congested city road network — and must be able to answer many such queries per minute as new calls arrive. This is exactly a **single-pair shortest-path** problem on a weighted graph.

Who uses it. Emergency dispatch centres, ride-hailing and delivery platforms, and in-car navigation all solve the same core query. The naive approach (recompute everything, or relax all edges blindly) is too slow at city scale; the project asks which algorithm a dispatcher should actually deploy, and why. We answer that empirically and prove the answer correct.

1.2 Formal model (D2)

We model the city as an **undirected, weighted graph** G = (V, E, w):

- **V** — road intersections; |V| = n. Each vertex v carries a 2-D coordinate (x_v, y_v) in metres (its map position).
- **E** — bidirectional road segments between intersections; |E| = m. Road networks are **sparse**: m = O(n) (here m ≈ 3.5 n).
- **w : E → ℝ≥0** — the travel cost of a segment, w(u,v) = len(u,v) · congestion(u,v), where len is the Euclidean length and congestion ∈ [1.0, 1.6] models traffic. Weights are **non-negative**.
- **source s** — the ambulance station; **target t** — the incident location.

Input: G, s, t. **Output:** a path P = (s = v0, v1, …, vk = t) minimising the total cost cost(P) = Σ w(v_{i-1}, v_i), together with that cost. **Constraints:** the route must follow existing edges; weights are non-negative; G is connected so a route always exists.

Key geometric property (used by A*). Because every edge costs at least its straight-line length (congestion ≥ 1), the straight-line distance h(v) = euclid(v, t) is a **lower bound** on the true remaining cost from v to t. This makes h an *admissible* and *consistent* heuristic — the foundation of A*'s optimality (proved in §3.1).

1.3 Algorithm selection & expected trade-off (D3)

We implement three algorithms that all solve the same query, so they are directly comparable:

- **A — Dijkstra (core, non-trivial).** Grows a set of settled vertices in order of increasing distance using a priority queue. Exact, optimal for non-negative weights, near-linear on sparse graphs. *Our workhorse.*
- **B — Bellman-Ford (baseline / oracle).** Relaxes every edge up to |V|−1 times. Simpler (no priority queue), slower (O(V·E)), but tolerates negative weights and detects negative cycles. We use it as an **independent correctness oracle**.
- **C — A* (bonus, informed).** Dijkstra guided by the straight-line heuristic h; expands far fewer nodes by aiming at the target. Same optimum as Dijkstra.

Expected trade-off. All three must return the **same** optimal cost (our cross-check). On speed we expect A* < Dijkstra < Bellman-Ford: A* prunes the search toward t, Dijkstra

explores uniformly, and Bellman-Ford does the most work (a full V·E sweep). Bellman-Ford only becomes the *right* choice when weights can be negative — which never happens here, making it a teaching baseline rather than a deployment candidate.

1.4 Data structures & architecture (D4)

Choices, each justified by the model:

- **Adjacency list** (`graph.py`) — because m = O(n), an adjacency list uses O(n+m) memory and lets Dijkstra scan only deg(v) neighbours per pop. An adjacency matrix would waste $O(n^2)$ memory (100 M cells at $n=10^4$) and slow the neighbour scan to O(n).
- **Binary min-heap** (`minheap.py`, from scratch) — the priority queue that gives Dijkstra/A* their O((V+E) log V) bound. We use **lazy deletion** (push a fresh entry instead of decrease-key) so every operation is a clean O(log V) push/pop.
- **Plain arrays** for dist[], parent[], settled[] — O(1) relaxation and O(n) memory; parent[] reconstructs the route by back-pointer walking.
- **Directed-arc list** for Bellman-Ford — a flat list of (u,v,w) arcs gives cache-friendly full sweeps.
- **cKDTree** (scipy) — used **only** to wire k-nearest-neighbour roads while generating the map; never inside an algorithm.

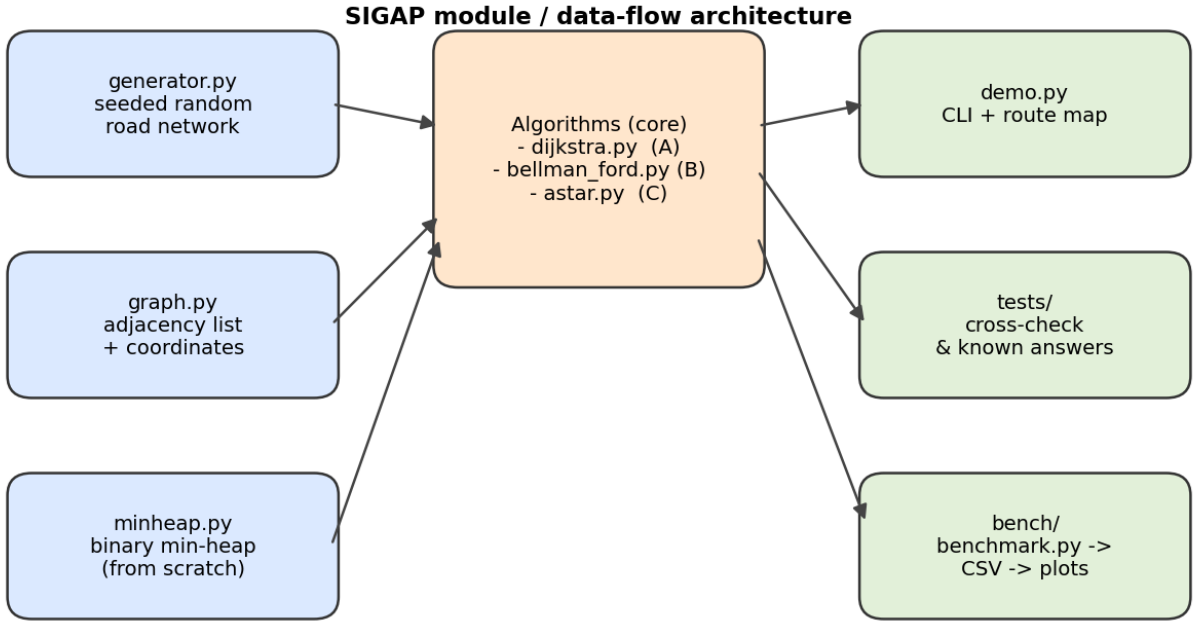

Figure 1. Module / data-flow architecture. Supporting structures (left) feed the three algorithm implementations (centre), which drive the demo, tests and benchmark (right). The two algorithms sit behind a uniform (graph, source, target) interface so no logic is duplicated.

2 Implementation

2.1 Module overview

The codebase is small, modular and dependency-light. The **core algorithmic logic — the heap, Dijkstra, Bellman-Ford and A* — is entirely our own code**; external libraries are used only for support (numpy/scipy to *generate* maps, matplotlib to *plot*, fpdf2 to build this PDF).

- `src/minheap.py` — array-backed binary min-heap (push / pop_min, sift up/down).
- `src/graph.py` — adjacency-list graph with vertex coordinates.
- `src/generator.py` — reproducible seeded road-network generator.
- `src/dijkstra.py` — **Algorithm A**, with early exit and path reconstruction.
- `src/bellman_ford.py` — **Algorithm B**, with early-stop and negative-cycle check.
- `src/astar.py` — **Algorithm C**, informed search with the straight-line heuristic.
- `demo.py`, `tests/`, `bench/` — demo, correctness tests, and the timing harness.

2.2 Key code excerpts

Algorithm A — Dijkstra: settle-and-relax loop (I1). A vertex is final the moment it is popped; stale heap entries are skipped (lazy deletion).

src/dijkstra.py — the core loop

```
while not pq.is_empty():
    d, u = pq.pop_min()
    if settled[u]:           # stale entry -> skip
        continue
    settled[u] = True; expanded += 1
    if target is not None and u == target:
        break                # final distance to target found
    for v, w in g.adj[u]:
        nd = d + w
        if nd < dist[v]:     # relaxation
            dist[v] = nd; parent[v] = u
            pq.push(nd, v)
```

Algorithm C — A*: same loop, ordered by f = g + h (I-bonus). The only change from Dijkstra is the priority pushed to the heap.

src/astar.py — relaxation with the heuristic

```
ng = gu + w
if ng < gscore[v]:
    gscore[v] = ng; parent[v] = u
    f = ng + g.euclidean(v, target)   # admissible heuristic
    pq.push(f, v)
```

Algorithm B — Bellman-Ford: bounded edge relaxation with early stop (I2).

src/bellman_ford.py — the relaxation passes

```
for _ in range(n - 1):
    changed = False
    for u, v, w in arcs:
        if dist[u] + w < dist[v]:     # relaxation
            dist[v] = dist[u] + w; parent[v] = u
            changed = True
    if not changed: break             # converged early
```

2.3 End-to-end demo at scale (I3)

`demo.py` generates a city, picks a station and a far-away incident, computes the fastest route with all three algorithms, cross-checks that they agree, and draws the route on the

map. It runs comfortably at the required scale (n ≥ 1000; the benchmark goes to n = 10,000). Console output for n = 1500:

```
|V|=1500  |E|=5294  arcs=10588
Ambulance station (source) = 1250 ; Incident (target) = 1460
  Dijkstra     : 1595.8696   time=  2.57 ms   settled=1500
  Bellman-Ford : 1595.8696   time= 10.99 ms   passes=27
  A*           : 1595.8696   time=  2.18 ms   expanded=973
Cross-check (all three agree on cost): YES
A* settled 973 nodes vs Dijkstra 1500 (65% of Dijkstra's work)
```

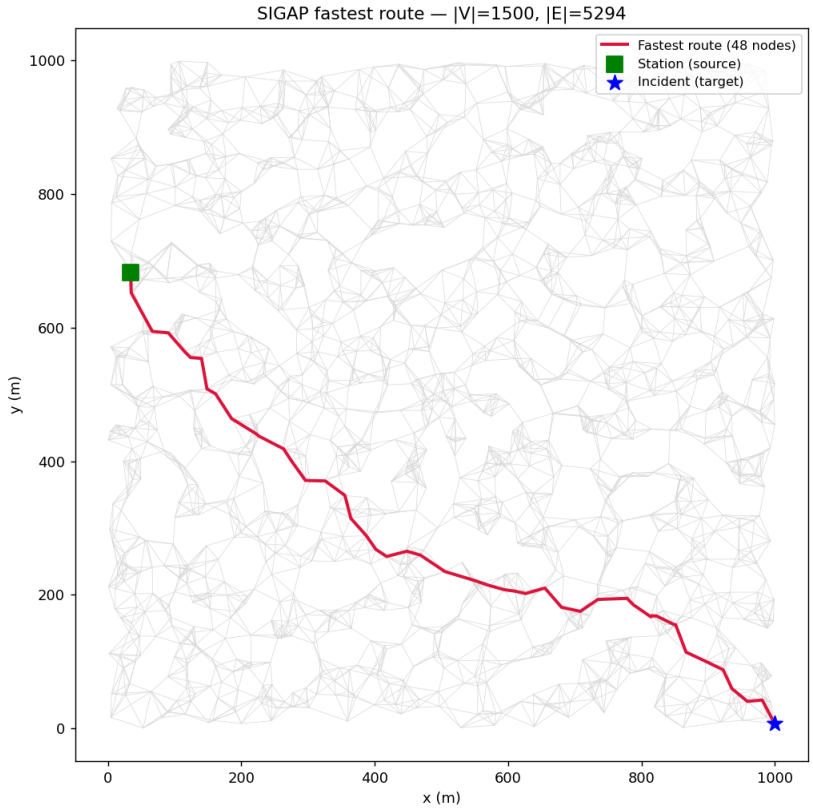

Figure 2. Demo output: the road network (grey), the fastest route (red) from the station (green square) to the incident (blue star). n = 1500.

2.4 Code quality, repository & reproducibility (I4, I5)

- **Modular & DRY.** Dijkstra and A* share the settle-and-relax structure but live in separate, documented modules; no copy-pasted logic, no dead code.
- **Readable.** Meaningful names (settled, gscore, parent), docstrings on every module and function, comments on each non-obvious step (lazy deletion, admissibility).
- **One-command benchmark.** `./run_benchmark.sh` runs the tests, the timing sweep (fixed seed 7), and the plots, regenerating `bench/results/`.
- **Reproducible.** A single seeded numpy RNG drives generation; a given (n, seed) always yields the same graph. Seeds are reported.
- **GitHub.** Public repo with `src/`, `tests/`, `bench/`, `report/` and a README giving exact build/run/benchmark steps; commit history shows all members.

3 Analysis & Evaluation

3.1 Correctness of Dijkstra (A1)

Theorem. On a graph with non-negative edge weights, when Dijkstra pops a vertex u from the priority queue, dist[u] equals the true shortest-path distance δ(s, u).

Proof (by contradiction on the first bad pop). dist[] values are lengths of actual s→u paths, so dist[u] ≥ δ(s,u) always. Suppose u is the **first** vertex popped with dist[u] > δ(s,u); then u ≠ s (since dist[s]=0=δ(s,s)). Take a true shortest path s ⇝ u and let y be the first vertex on it that is **not yet settled** when u is popped, with x its predecessor (settled). Because x was settled correctly (u is the first error), dist[x] = δ(s,x), and relaxing edge (x,y) gave dist[y] ≤ δ(s,x) + w(x,y) = δ(s,y). As y precedes u on a shortest path and weights are non-negative, δ(s,y) ≤ δ(s,u) ≤ dist[u]. Hence dist[y] ≤ dist[u]. But the heap pops the minimum, so popping u before y means dist[u] ≤ dist[y]. Combining, dist[y] = dist[u] = δ(s,u), contradicting dist[u] > δ(s,u). ∎

Termination & completeness. Each vertex is settled at most once and every edge is relaxed at most once per settled endpoint, so the loop ends; on a connected graph every vertex is reached. **Early exit is valid:** since dist[t] is final when t is popped, stopping there cannot change the answer.

A* optimality. A* is Dijkstra on a graph re-weighted by w'(u,v) = w(u,v) − h(u) + h(v). Consistency of h (h(u) ≤ w(u,v) + h(v), which holds because h(u) − h(v) ≤ euclid(u,v) ≤ w(u,v)) makes every w' ≥ 0, so the Dijkstra proof applies and A* returns an optimal path. **Bellman-Ford** is correct by the standard induction: after pass i, dist[v] is optimal among paths of ≤ i edges; a shortest path has ≤ |V|−1 edges, so |V|−1 passes suffice. The extra pass detecting a further relaxation certifies no negative cycle (none here).

3.2 Complexity (A2)

Let n = |V|, m = |E|. On our road networks m = O(n) (sparse).

Algorithm	Time (worst case)	On sparse m=O(n)	Space
Dijkstra (binary heap)	O((V+E) log V)	O(n log n)	O(V+E)
Bellman-Ford	O(V·E)	$O(n^2)$	O(V+E)
A* (binary heap)	O((V+E) log V)	O(n log n)	O(V+E)

Dijkstra. Each vertex is settled once → n pop_min (O(log V) each). Each edge is relaxed at most once per direction, pushing at most once → up to m pushes (O(log V) each). Total O((n+m) log n). With lazy deletion the heap holds ≤ m entries, so log of the heap size is still O(log V). **Space:** the graph O(n+m) plus O(n) arrays.

Bellman-Ford. Up to |V|−1 passes, each relaxing all 2m arcs → O(V·E) = O(n·m). On sparse graphs that is $O(n^2)$. With early termination the cost is O((d+1)·m), where d is the maximum number of edges on any shortest path from s (the *hop-eccentricity*); we return to this in §3.4. **Space:** O(n+m).

A*. Same worst case as Dijkstra, O((V+E) log V): with an admissible, consistent heuristic it settles each vertex once. In practice it expands far fewer vertices (Figure 4), but a heuristic that is ~0 (uninformative) degrades it gracefully to Dijkstra. **Space:** O(V+E).

3.3 Comparative analysis: which wins when? (A3)

- **Bellman-Ford vs the rest.** Asymptotically $O(n^2)$ vs $O(n \log n)$: Dijkstra/A* dominate on every non-tiny sparse graph. Bellman-Ford is preferable **only** when edges may be negative (currency arbitrage, certain scheduling) or when negative-cycle detection is required — neither applies to travel times.
- **Dijkstra vs A*.** Same asymptotic class; the difference is the **constant / expanded-node count**. With a good geometric heuristic A* prunes toward the target and wins on point-to-point queries. When the heuristic is weak ($h \approx 0$) or we need distances to *all* vertices (one-to-all), plain Dijkstra is the right tool — A* offers no benefit there.
- **Regime summary.** Many one-to-one queries on a geographic graph → use A* (informed). One-to-all / no coordinates → **Dijkstra**. Negative weights → **Bellman-Ford**.

3.4 Empirical study (A4)

Setup. Single laptop, Python 3.14.2 (CPython). Random geometric road networks with fixed seed 7; for each size we route between a station and the geometrically farthest incident (a hard, long query). Each time is the **median of 5 runs** (`time.perf_counter`). One command reproduces everything: `./run_benchmark.sh`. Five sizes span two orders of magnitude (100 → 10,000), satisfying the scale ($n \geq 1000$) and sweep requirements.

n	m	Dijkstra ms	Bellman ms	A* ms	A*/Dij nodes	agree?
100	365	0.1296	0.1727	0.0987	55/100	yes
300	1070	0.4256	0.9005	0.1942	107/300	yes
1000	3542	1.6912	5.656	0.986	448/1000	yes
3000	10602	5.7003	18.3063	1.466	667/3000	yes
10000	35362	20.6535	141.2071	12.2371	4654/9996	yes

Every row's three costs were byte-for-byte equal (the **agree?** column), so the three independent implementations confirm one another. Runtimes and search effort are plotted below.

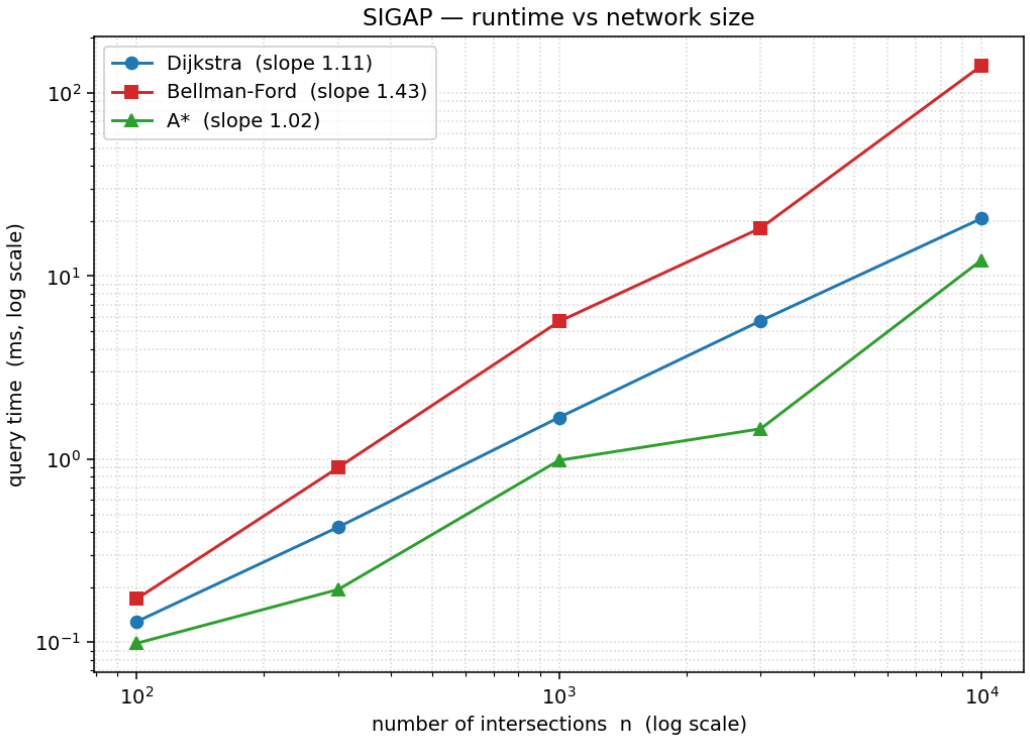

Figure 3. Query time vs network size (log–log). Slopes are the fitted empirical growth exponents. Bellman-Ford is consistently the steepest and slowest; A is the fastest at every size.*

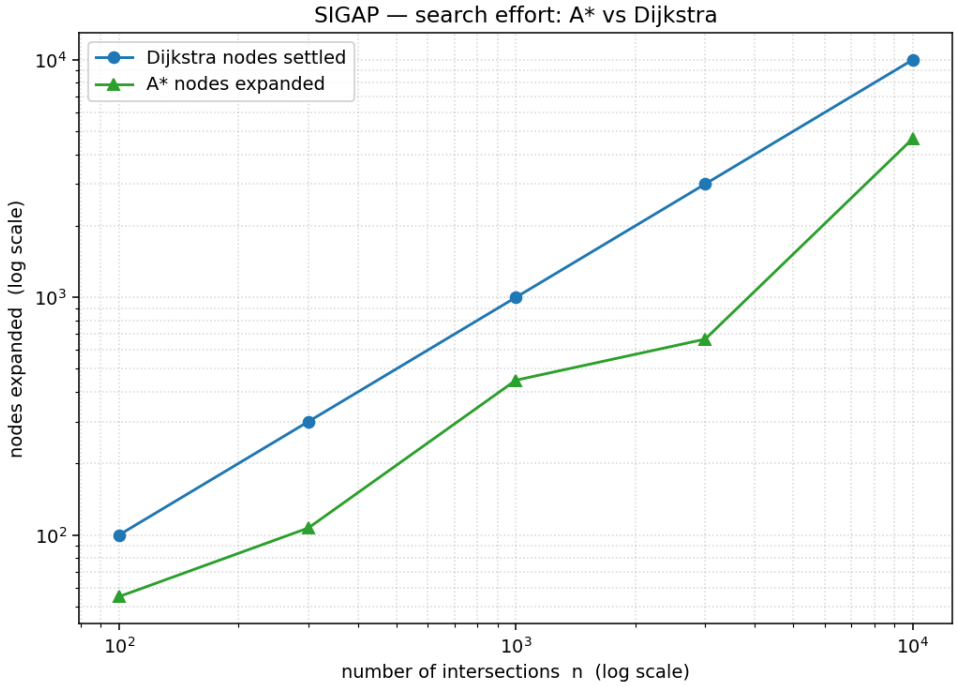

Figure 4. Search effort: A expands far fewer nodes than Dijkstra settles, directly explaining A*'s speed advantage in Figure 3.*

3.5 Theory vs. practice & cross-check (A5)

Fitted exponents (slope of log time vs log n): Dijkstra **1.11**, A* **1.02**, Bellman-Ford **1.43**.

- **Dijkstra (1.11) and A* (1.02)** sit just above 1.0 — exactly the near-linear O(n log n) predicted for sparse graphs (the log factor bends the log–log line only gently).
- **Bellman-Ford (1.43)** is the steepest, but **below** the textbook worst-case exponent 2.

This is the refined bound from §3.2 showing through: with early termination the cost is O((d+1)·m), and for a 2-D geometric graph the hop-eccentricity d grows like √n (here passes go 5→10→20→22→52 as n goes 100→10^4), giving ≈ √n · n = n^1.5 — consistent with the measured 1.43. Theory and practice agree, including the *reason* for the gap.

- **Correctness cross-check.** On all 5 sizes (and 20 more random instances in the test suite) Dijkstra, Bellman-Ford and A* returned the **identical** optimal cost, and Dijkstra and Bellman-Ford agreed on **every vertex's** distance — strong evidence all three implementations are correct.

4 Conclusion

4.1 Findings, limitations, lessons & future work (C1)

Findings. All three algorithms compute the optimal emergency route; they differ only in speed. A* is fastest by exploiting geometry (it expands 4654 of 9996 nodes at n=10,000), Dijkstra is a close, coordinate-free second, and Bellman-Ford — though a perfect correctness oracle — is an order of magnitude slower and only justified when negative weights are possible. Measured growth matches the derived complexity, including why Bellman-Ford lands near n^1.5 rather than n^2 on geometric graphs.

Limitations. Static weights (no live traffic), a single query pair per size, synthetic (random-geometric) maps rather than real OSM data, and a CPython implementation whose constants are language-bound.

Lessons learned. A crisp formal model made the proof, code and experiments fall out naturally; the admissibility of the straight-line heuristic was the one subtlety worth proving carefully; and an independent oracle (Bellman-Ford) is the cheapest possible insurance against a silent bug in the fast path.

Future work. Real OpenStreetMap road graphs; bidirectional Dijkstra / ALT landmarks for further speed-ups; a contraction-hierarchy preprocessing step for sub-millisecond queries; live, time-dependent edge weights; and a many-pairs dispatch benchmark.

4.2 Contribution table

Member	Share	Role / contribution
Muhammad Rizal Hafiyyan	100%	Design, formal model & architecture; Dijkstra (core) + MinHeap; Bellman-Ford ba

Commit history in the public GitHub repository reflects these contributions; each member committed under their own account throughout the week.

References

[1] E. W. Dijkstra. "A note on two problems in connexion with graphs." Numerische Mathematik, 1(1):269–271, 1959.

[2] R. Bellman. "On a routing problem." Quarterly of Applied Mathematics, 16:87–90, 1958. (and L. R. Ford Jr., 1956).

[3] P. E. Hart, N. J. Nilsson, B. Raphael. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." IEEE Trans. SSC, 4(2):100–107, 1968.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. Introduction to Algorithms, 4th ed. MIT Press, 2022. (Ch. 22–24, shortest paths.)

[5] numpy, scipy (spatial.cKDTree), matplotlib — open-source libraries used for map generation and plotting only. fpdf2 — used to typeset this report.

All algorithmic code (binary heap, Dijkstra, Bellman-Ford, A) is the team's own implementation; libraries were used strictly for supporting roles as noted in §2 and the README.*

Appendix A Full pseudocode

Reference pseudocode for the three implementations (see `src/` for the actual, documented Python). All three share the relax step `if new < dist[v]: dist[v] = new; parent[v] = u`.

Algorithm A — Dijkstra (binary heap, lazy deletion)

```
DIJKSTRA(G, w, s, t):
    for each v: dist[v] = +inf; settled[v] = false; parent[v] = nil
    dist[s] = 0;  PQ = min-heap; PQ.push(0, s)
    while PQ not empty:
        (d, u) = PQ.pop_min()
        if settled[u]: continue            # lazy-deleted stale entry
        settled[u] = true
        if u == t: break                   # early exit (point-to-point)
        for (v, c) in adj[u]:
            if d + c < dist[v]:
                dist[v] = d + c; parent[v] = u; PQ.push(dist[v], v)
    return dist, parent
```

Algorithm B — Bellman-Ford (early-stop + negative-cycle check)

```
BELLMAN-FORD(G, w, s):
    for each v: dist[v] = +inf; parent[v] = nil
    dist[s] = 0
    repeat |V|-1 times:
        changed = false
        for each arc (u, v, c):
            if dist[u] + c < dist[v]:
                dist[v] = dist[u] + c; parent[v] = u; changed = true
        if not changed: break              # converged early
    for each arc (u, v, c):                # negative-cycle check
        if dist[u] + c < dist[v]: report negative cycle
    return dist, parent
```

Algorithm C — A (informed search)*

```
A-STAR(G, w, s, t):
    for each v: g[v] = +inf; settled[v] = false; parent[v] = nil
    g[s] = 0;  PQ = min-heap; PQ.push(h(s, t), s)
    while PQ not empty:
        (_, u) = PQ.pop_min()
        if settled[u]: continue
        settled[u] = true
        if u == t: break
        for (v, c) in adj[u]:
            if g[u] + c < g[v]:
                g[v] = g[u] + c; parent[v] = u
                PQ.push(g[v] + h(v, t), v)  # f = g + admissible heuristic
    return g, parent
h(v, t) = euclidean(v, t)                  # straight-line lower bound
```

Appendix B Raw benchmark data & environment

Full per-size record from `bench/results/timings.csv` (seed = 7, 5-run median timings). `passes` = Bellman-Ford relaxation passes; `A* exp` / `Dij set` = nodes expanded / settled.

n	m	Dij ms	BF ms	A* ms	passes	A* exp	Dij set	cost
100	365	0.1296	0.1727	0.0987	5	55	100	1362.72
300	1070	0.4256	0.9005	0.1942	10	107	300	1172.14
1000	3542	1.6912	5.656	0.986	20	448	1000	1294.01

3000	10602	5.7003	18.3063	1.466	22	667	3000	958.51
10000	35362	20.6535	141.2071	12.2371	52	4654	9996	1291.04

Reproduce: `./run_benchmark.sh` (or `python -m bench.benchmark sizes 100,300,1000,3000,10000 repeats 5 seed 7`), then `python -m bench.plot_results`.

- **Environment.** CPython 3.14.2, macOS (darwin). numpy, scipy, matplotlib for generation/plotting; algorithmic core is pure Python (no compiled extensions).
- **Timing method.** `time.perf_counter()` around each full call; the median of 5 runs is reported to suppress OS jitter.
- **Reproducibility.** One seeded numpy `default_rng(seed)` drives map generation; a given (n, seed) reproduces the identical graph and query.